Constructing objectKV: A Transactional KV Kernel with Stable Logs, Disposable Serving Profiles, and Immutable Objects

Wiley Jones
DOSS
New York, USA
wiley@doss.com

ABSTRACT

This paper describes the construction and current evidence boundary of objectKV, an ordered transactional key-value kernel that separates the commit path from permanent object-resident state. The target system acknowledges a commit after a replicated stable-media transaction log is durable, serves active key ranges from bounded SSD-resident or RAM-resident profiles, and asynchronously publishes immutable row and analytical objects. Its load-bearing recovery equation is $Database(C) = ObjectState(O) + txLog(O, C]$, where C is the cell commit version and O is the object-durable version. The design keeps object operations off the normal commit and resident-read paths while retaining object storage as the portable, reconstructable base.

We report a construction program rather than a completed database. The current implementation verifies deterministic MVCC, commit, recovery, publication, garbage-collection, and exact HTAP overlay contracts. It also verifies local stable-file reopening, a pinned MinIO S3-compatible authority profile, real OpenRaft operating-system processes, TCP communication, process kill, leader failover, unknown object outcomes, and empty-scratch publisher replacement. Most receipts remain local-system results rather than independent-failure-domain claims. One bounded private Google Cloud run now verifies the complete single-range recovery and resident-read mechanism over regional GCS, a persistent SSD authority volume, and local NVMe serving scratch. An engine-native experiment corrected value ownership but compared a six-process recovered candidate with a bare RocksDB control. It passed throughput and missed p99. A topology-matched rerun then retained 90.89% and 91.97% of direct RocksDB throughput; p99 was 0.913x control in both process orders. At 8 and 32 synchronized clients, the native boundary retained 87.34% through 89.06% of control throughput and kept p99 between 1.107x and 1.184x control. All eight concurrent results passed their explicit constraints, issued zero measured object operations, and emitted correlated OTel signals. These results admit a single-range resident-read mechanism, not a complete cell. A live semantic eliminator rejected TiKV as the strict-serializability control because both transactions in the frozen write-skew history committed. FoundationDB rejected the same write skew and remains the semantic oracle and fallback profile. A frozen FoundationDB plus GCS logical-lifecycle batch reconstructed 950 rows into an empty logical generation with the exact source digest, fenced an old generation transaction, and discarded three negative controls. A second bounded cloud run then deleted the source FoundationDB VM, boot disk, and provider SSD before reconstructing the same 950-record digest on a fresh cluster from two named GCS objects. Its formal positive passed 16 hard gates; the same-cluster hidden-media control was discarded. The exact source, machines, evaluator binary, and OTel logs, metrics, and traces are recorded. This proves object-closure sufficiency for one physical media-loss case, not HA or a performance curve. A subsequent clean local-process contract composed three-process generation and publication authorities, one authority failover, source fencing, destination activation, and stale commit, route, and publication checks. Its positive received `keep` with zero anomalies; the three-surface stale-source control received `discard`. This verifies the compound-fence process mechanism, not yet a resurrected FoundationDB provider. The native RocksDB and OpenRaft plane remains the primary bounded research lane; objectKV retains FoundationDB as a comparison and fallback rather than making it the product boundary. Multiple machines, multiple zones, RAM admission, multi-range transactions, and sustained capacity curves remain proposed or are being evaluated. The paper defines the proof ladder and falsifiable curves that must clear before objectKV can support PostgreSQL, log-oriented abstractions, and version-aligned DataFusion analytics.

PVLDB Reference Format:
Wiley Jones. Constructing objectKV: A Transactional KV Kernel with Stable Logs, Disposable Serving Profiles, and Immutable Objects. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/Doss-com/objectKV.

1 INTRODUCTION

Cloud databases commonly separate compute from object storage, but they still need a low-latency durability path and a read path that does not turn every point lookup into an object request. Systems including Aurora, Neon, and Socrates separate durable logging, page service, and compute to different degrees [1, 10, 16]. FoundationDB instead exposes a small ordered transactional API over a distributed transaction system and storage fleet [18]. TiDB combines a Raft-backed row path with analytical replicas [7]. These systems establish important pieces, but none is exactly the primitive considered here: an open, value-native, FoundationDB-like kernel whose permanent reconstructed base is a portable set of immutable objects, whose hot path can use bounded SSD or RAM

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

serving state, and whose one version history can feed row and column layouts.

objectKV explores that primitive. It does not attempt to make object storage the normal write-ahead log. Object latency and request economics make that path a poor fit for latency-sensitive, multi-writer transactions. Instead, the design places consensus and commit durability on replicated stable media. NVMe is the first profile, but the durability contract does not require a serving worker's local device. Object storage is the permanent, reconstructable base and the location of immutable row, analytical, branch, snapshot, and retained-history artifacts. Serving workers are disposable. They may keep complete assigned ranges in SSD or RAM, but no worker-local serving image is the only copy of acknowledged data.

The resulting architecture is intentionally closer to a TiKV or FoundationDB hot path than to an object-only LSM tree. Its differentiator is not that it removes local storage. The differentiator, if verified, is that local storage becomes bounded serving and recovery state rather than a permanent full replica of the database. This separation could make branching, rapid placement, independent compute scaling, open-format analytical materialization, and empty-disk recovery cheaper and more portable. It could also fail. Metadata amplification, cold-read latency, objectification debt, double logging, or a global transaction bottleneck could eliminate the advantage.

This paper makes five contributions:

- It states a bottom-up construction of a transactional object-native KV kernel, from an ordered log algebra through consensus, serving images, immutable publication, and exact analytical overlays.
- It separates code presence from measured evidence through the statuses Code Complete, Verified, and Evaluating.
- It reports the exact systems substrate behind current receipts and identifies where the program still uses deterministic models, one-machine files, loopback services, or local processes.
- It defines go/no-go curves against RocksDB, TiKV-like Raft storage, DataFusion, and PostgreSQL controls so the project can be stopped if its claimed physical or operational advantage does not materialize.
- It defines one dependency-ordered golden scenario across kernel, durability, object, serving, distribution, consumer, HTAP, and economics surfaces without collapsing their distinct claims into a blended score.

2 SCOPE AND STATUS SEMANTICS

2.1 Kernel boundary

The kernel owns ordered byte keys, opaque byte values, versions, bounded transactions, range reads, conflict checking, commit outcomes, retention, and recovery. PostgreSQL pages, logical rows, secondary indexes, inverted postings, Redis data structures, SQL triggers, and schema semantics are serving-model concerns above that waist. This separation lets the same kernel pressure-test several database shapes without importing one consumer's semantics into the transaction protocol.

A cell is one complete transaction, durability, serving, and recovery cluster. A tenant database is the normal transaction domain, so one bounded transaction may span its ranges inside a cell. Cells have independent versions and do not offer cross-cell atomicity. A future metacluster may place and move tenants, but it does not merge transaction histories.

```
fleet
+-- metacluster              [FUTURE]
+-- cell A
|   +-- tenant databases
|   +-- transaction system
|   +-- range groups
|   +-- object state
+-- cell B
```

Figure 1: Fleet ownership hierarchy. A cell is the transaction and recovery boundary, not a page, shard, or object.

2.2 Proof status

objectKV uses three active implementation states. Code Complete means the path is present and code gates pass; it implies no performance or operational result. Verified requires an exact revision, suite hash, profile, backend, immutable seeds, run identifier, primary metric, hard gates, negative controls when applicable, and telemetry receipt. The claim applies only to that named scope. Evaluating means a required curve or infrastructure receipt is in progress. Proposed and future work remain explicitly labeled.

The distinction prevents a deterministic model result from silently becoming a production system claim. A local filesystem recovery experiment may be verified for ordinary files on one machine while independent disks, hosts, and zones remain unverified.

3 SYSTEM MODEL AND INVARIANTS

Let C be the latest cell commit version, O the latest object-durable version, and W_p the analytical base watermark of table partition p. For a query target T, the required relationships are

$$O \le C, \qquad W_p \le T \le C. \tag{1}$$

The authoritative database at C is

$$Database(C) = ObjectState(O) + txLog(O, C]. \tag{2}$$

Every acknowledged mutation newer than O must therefore remain in the replicated *txLog*. Records through version X may be reclaimed only after authoritative object metadata and immutable bytes reconstruct the database through X. The *txLog* is not an archival application log. It is a bounded recovery suffix whose capacity and outage policy define the cell's continued write envelope.

The analytical snapshot of partition p is

$$Table_p(T) = ColumnarBase_p(W_p) + RowChanges_p(W_p, T]. \tag{3}$$

The analytical tail may need longer retention than the recovery suffix. Query freshness is T, not W_p. Lag changes the amount of merge work rather than the logical snapshot.

The system assumes that object bytes are immutable once published, object LIST results are never authoritative, a lost object response is an unknown outcome, and mutable publication roots

require fenced conditional updates or a separate transactional authority. A successful commit response means quorum durability in the declared *txLog* topology, not immediate object durability.

4 BOTTOM-UP CONSTRUCTION

Figure 2 shows the target composition and the present evidence boundary.

4.1 Ordered log substrate

`okv-log` is a pure partition-local state machine over opaque records. It owns append planning, suffix replacement, prefix purge, exact and clamped reads, and replay semantics. It deliberately has no filesystem, checksum, consensus, object, or transaction meaning. This narrow algebra can underlie multiple durable structures without coupling them to a physical frame format.

`okv-wal` layers stable-storage and consensus metadata policy on the log. It owns versioned `OKVR` and `OKVW` frames, checksums, synchronization, votes, committed identities, quorum evidence, and acknowledgement policy. The separation is useful for two additional abstractions. `okv-log` can become a transactionally emitted retained application stream with cursors and retention, while `okv-wal` remains a short recovery structure. PostgreSQL WAL adaptation can then use the general log substrate without making PostgreSQL record semantics part of the kernel.

The log algebra is Code Complete. Its per-node integration and raw accepted and rejected histories are code complete, while a fully verified standalone log performance and compatibility matrix is still required.

4.2 Commit and consensus

The bootstrap commit envelope freezes request identity, read version, conflict ranges, ordered mutations, required log tags, generation, and durability. The current implementation includes a pinned OpenRaft per-node storage adapter and a three-node protocol. Deterministic network execution covers election, quorum commit, leader isolation, successor election, stale-suffix replacement, node bounce, and catch-up. A real-process lane adds TCP, operating-system processes, lost replies, process kill, leader failover, and durable request-outcome retry.

This is not yet a complete transaction cell. Read-version assignment, scalable conflict resolution, direct routed reads, range groups, multi-range atomicity, and a bounded control-root recovery path remain separate work. Raft is the initial replication choice because its leader, term, and log mechanics map cleanly to the executable test harness and broad implementation ecosystem. The architecture does not require claiming that Raft is universally superior to Paxos. The more important decision is whether replication is per cell or per range. The target is range-local consensus with cell-level transaction atomicity, since one global Raft group would cap throughput while one group per cell would make range placement mostly cosmetic.

4.3 Disposable serving profiles

The proposed serving worker owns an assignment, not permanent bytes. Every profile keeps a recent MVCC overlay in RAM, a selected *ServingImage*, indexes into immutable row objects, cached object blocks, and enough state to answer reads at admitted versions. The `ssd_resident` profile uses a local NVMe image. RocksDB is its first candidate because it provides a mature ordered embedded engine, block cache, prefix and range access, compaction, and well-understood controls. The `ram_resident` profile keeps its serving image in memory under a hard byte budget and spill policy. Neither image is the authority. A new worker must be able to reconstruct from an object base plus the retained *txLog* suffix.

Serving and durability profiles are orthogonal. A RAM serving worker may still acknowledge only after a stable replicated *txLog* quorum, called `ram_durable`. A proposed `ram_turbo` profile may return a weaker `BUFFERED` outcome before stable durability, but must never call that outcome `COMMITTED`. Moving a range between SSD and RAM profiles requires a generation-fenced handoff so stale workers cannot serve or publish after reassignment. The RAM profile becomes a product surface only if its end-to-end improvement clears the proposed 20% gate after identical correctness and durability controls.

Resident point reads should use the local image and issue zero object requests after warmup. A cold point read resolves a bounded manifest path, range-GETs an indexed object block, validates identity and integrity, installs the block or range, applies the required overlay, and returns the requested version. The database may grow without every worker retaining every byte because assignment and cache limits are explicit. If stable p99 requires every serving worker to hold a durable complete database copy, the central object-native thesis is rejected.

This layer is Evaluating. The public `SingleRange` path can activate a complete verified GCS object base into a bounded RocksDB image, apply the newer *txLog* suffix, and serve exact points after empty-worker replacement. A clean frozen-source R0 run used one private `n2-standard-8`, a named 375 GiB local NVMe SSD, regional GCS, and a separate OTel collector. Across 15 samples the public path reached 516,973 reads/s median and 2,482 ns median p99; direct RocksDB on the same machine reached 702,142 reads/s and 1,749 ns. Both issued zero measured object operations and returned zero incorrect reads. The mechanism and comparison are verified in this scope, but the public path is 26.37% worse on throughput and 41.91% worse on p99, so the 20% GP3.1 performance envelope is not admitted. The comparison engine enforced the throughput constraint; the p99 ratio was recomputed from the receipts against the declared threshold. The next run must make that secondary constraint executable. The 4 MiB working set isolates warm software overhead and does not establish an SSD cache-pressure curve.

A second frozen source made the p99 ratio executable and tested one variable: complete-image reads bypass manifest location and object-reference cloning. Candidate throughput improved 11.18% over the prior clean result. Candidate then control measured 575,498 versus 713,304 reads/s and 2,490 versus 1,841 ns p99. Control then candidate measured 573,999 versus 717,362 reads/s and 2,427 versus 1,867 ns p99. Throughput entered the 20% envelope in both orders; p99 remained 1.353x and 1.300x control. Every identity, mechanism, and telemetry check passed. The result triggers the predeclared stop condition for incremental wrapper optimization.

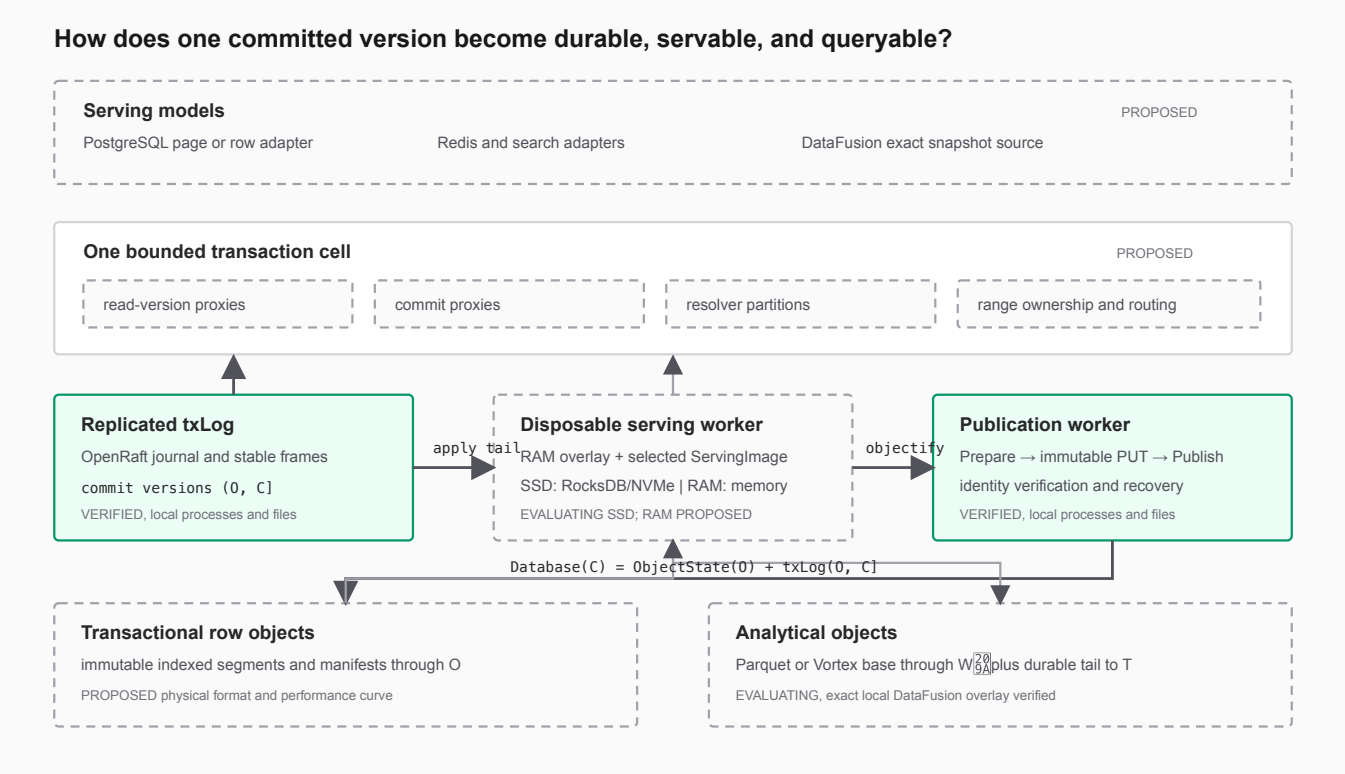

Figure 2: Bottom-up construction of the target system. Solid green boxes have measured local evidence. Dashed boxes remain proposed or are being evaluated. The current receipts do not yet form one complete production cell.

The next subject verifies generation, coverage, and closure at activation and frontier transitions, materializes the visible *txLog* suffix into the resident engine, and uses the engine-native point path.

That final subject is now implemented and measured. Its RocksDB namespaces hold the latest head, historical versions, and resident metadata. One atomic engine batch applies data and advances the visible frontier. A first run found thirty anomalies because object first and last keys were incorrectly used as serving range bounds; separating object closure span from authority-owned assigned range removed the anomalies. In the first candidate-then-control order, native and control measured 589,717 and 701,119 reads/s with 2,226 and 1,839 ns p99. In the reverse order they measured 587,199 and 710,184 reads/s with 2,261 and 1,778 ns p99. Throughput passed twice; p99 was 1.210x and 1.272x control and failed twice. That control did not pay for the native subject's recovered six-process authority topology.

A fourth frozen run placed native and direct owned-value RocksDB reads inside the same recovered topology. Native retained 0.9089x and 0.9197x control throughput in opposite process orders. Its p99 was 0.9134x and 0.9132x control. All four workloads passed sixteen hard gates and emitted correlated OTel logs, metrics, and traces. This admits the single-range native snapshot boundary, not replicated commit or a complete cell.

The concurrent runner partitions one exact read budget behind shared ready and start barriers. On clean GCP R0, the 8-client AB/BA native-to-control throughput ratios were 0.8798x and 0.8734x; p99 ratios were 1.1842x and 1.1220x. At 32 clients, throughput ratios were 0.8803x and 0.8906x; p99 ratios were 1.1072x and 1.1478x. The eight results contain 120 samples and 24 million measured reads. All 128 workload gates passed, every hot-read window issued zero object operations, and every run ID occurs in OTel logs, metrics, and traces. Cache pressure remains isolated into a later gate with explicit cache sizing and a reusable larger-than-cache fixture.

4.4 Objectification and publication

Objectification packs many committed mutations into immutable indexed row segments. It does not create one object per transaction, page, or key. A publisher first commits a publication intent, writes digest-addressed data and manifest objects, verifies the complete named closure, and then advances a fenced authoritative root. Unknown PUT outcomes are resolved by reading and verifying the intended object identity. LIST is useful for discovery and reclamation hints, never for deciding visibility.

The publication implementation verifies lost data-object, manifest, delete, and root-update responses. It verifies replacement of a killed publisher with empty scratch state and recovery of the original root outcome after killing the accepting authority leader. Garbage collection uses transactional in-flight roots, complete reachability marks, quarantine, delete-time root revalidation, and per-digest reservations so a sweep cannot delete an object being published.

Current evidence uses an Apache `object_store` client against local files and a separate local OpenRaft authority. The MinIO conformance lane uses a real S3-compatible server, but the end-to-end publisher lane has not yet run against GCS or independent hosts. Multipart residue, repeated unknown-response budgets, publisher abandonment, sweeper takeover, and generation-bound effect fencing remain open.

4.5 Exact analytical overlay

The physical HTAP prototype pins DataFusion 54.0.0 and Arrow/Parquet 58.3.0. A custom table provider reads a Parquet base and an Arrow change tail. A streaming execution plan validates ordering across record-batch boundaries, starts from the minimum relevant partition watermark, reduces changes by logical identity, preserves deletion invalidation, and emits an ordered snapshot at one target version. The admitted local fixture buffered at most four rows and 5,518 bytes, materialized no complete input, spilled no bytes, and produced exact results in all three frozen seeds.

The result is VERIFIED operator evidence, not a complete HTAP system. Version-bound manifests and leases, multiple execution ranges, partition-aware pruning, full-query memory, larger data, and the $T - W_p$ cost curve remain EVALUATING. The overlay also cannot safely push a predicate that would hide a tail key needed to suppress an older base row.

5 TRANSACTION AND READ PIPELINES

5.1 Commit pipeline

A target one-range transaction proceeds as follows:

(1) A client obtains a cell read version and reads assigned ranges from serving workers.
(2) The commit proxy freezes request identity, conflict ranges, mutations, range participants, and generation.
(3) Resolver partitions validate read and write conflicts.
(4) The transaction receives one commit version and its mutations are sent to every required range *txLog* group.
(5) A response is successful only after the declared quorums synchronize the exact record and the durable request outcome is reconstructable.
(6) Serving workers apply the committed tail. Publishers later advance O, after which eligible *txLog* prefixes can be reclaimed.

```
client
  -> read version + reads
  -> conflict validation
  -> commit version C
  -> txLog quorum sync
  -> durable outcome
  -> response

object PUT: absent
```

Figure 3: Minimal commit-path map. Object storage is not on the normal latency path.

Object storage is absent from steps one through five. Its availability affects objectification lag, cold misses, rebuilds, and eventually backpressure. It does not add normal commit latency. During a prolonged object-store brownout, the cell may continue committing only within a declared retained-log budget. It must rate-limit and eventually refuse writes before that recovery suffix becomes unbounded.

5.2 OLTP read pipeline

A resident read routes directly to a serving worker for the key range. The worker checks its RAM overlay and local indexed image, then returns the newest visible value at the requested version. A nonresident read follows bounded metadata to an immutable object block, range-GETs and verifies it, installs it in cache, overlays newer mutations, and returns. Range scans merge sorted local state and immutable blocks rather than performing one object request per key.

```
key + snapshot T
  -> RAM overlay
  -> selected ServingImage
       +-- SSD: RocksDB/NVMe
       +-- RAM: ordered memory image
  -> bounded object-block cache
  -> bounded manifest lookup
  -> object range GET
```

Figure 4: Point-read fallthrough. A resident hit ends in the selected serving profile; lower stages are the explicit elastic miss path.

PostgreSQL can initially map relation pages to opaque KV values so upstream PostgreSQL keeps parser, planner, executor, catalog, heap, index, constraint, trigger, and extension behavior. That adapter still faces a hard WAL decision. Keeping both PostgreSQL WAL and OBJECTKV durability may double logging and writes. Suppressing or adapting PostgreSQL WAL moves more recovery responsibility into the storage interface. The first bridge must account for every byte and sync at both layers rather than hiding the amplification.

5.3 OLAP read pipeline

A query chooses target T and pins its manifests and change retention. For each partition, DataFusion reads a columnar base through W_p and the durable change tail through T. A sorted primary-key merge suppresses base rows touched by the tail and emits the latest nondeleted row. Different partitions may have different watermarks, but every table provider uses the same T. The query lease pins immutable inputs without holding a long OLTP transaction open.

This is a shared logical history, not one physical format. Row objects optimize transactional point and range access. Parquet or Vortex artifacts optimize column scans. The common contract is versioned logical state, schema history, immutable identities, and fenced publication.

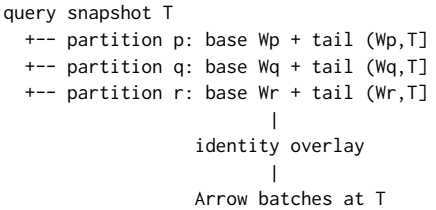

Figure 5: Exact analytical hierarchy. Watermarks may differ while the logical query version remains one value T.

6 CURRENT IMPLEMENTATION EVIDENCE

Table 1 records selected admitted receipts. Durations reported for fault suites are whole harness executions, not client latency. The named GCP R0 admission suite exports bounded-cardinality OTel logs, traces, and metrics. Earlier local suites do not all have complete OTel evidence.

A paired developer application path compares materialized KV history with versioned application deltas over the real `okv-model` and `okv-log` crates. Tetris generates long high-rate histories; Chess makes snapshots, historical forks, divergent suffixes, and named branch switching readable. The GP-G0 through GP-G6 proof ladder admits byte comparisons only when both representations finish at identical state fingerprints. Separate receipts then prove transactional alignment, canonical envelopes through three OpenRaft processes on one host, disposable RAM serving images, recursively verified immutable-object publication, copy-on-write branches, and reserved garbage collection. These are bounded compositions rather than one production cell: the object lanes use an in-memory adapter and pure publication authority, and the process lane has no independent host durability. A separate GCP R0 receipt exercises regional GCS and local-NVMe SSD serving on one private runner. It verifies that mechanism but does not admit its performance or prove independent-host durability. GP2.5.3 separately uses sequential source and restore VMs to verify reconstruction after complete source-provider media deletion. The GP2.5.4 local process rung separately verifies the external generation, routing, and publication decisions. A dual-provider GCP harness is code complete but has not executed.

The most consequential incumbent result is a SlateDB local-filesystem scale curve. At 1 MiB and 8 MiB, fresh reopen reached the first correct read in 4.85 ms and 6.19 ms. At 64 MiB it took 424.1 ms and read 210,773,938 bytes before the first correct read. The program therefore stopped that incumbent configuration. The result does not reject object-resident storage as a category. It rejects a configuration whose reopen work grew toward a full-state scan rather than bounded metadata plus requested blocks.

The objectKV-owned pilot tests that bounded alternative. Its G4.1 diagnostic kept an exact cold point read to one indexed data range GET and approximately 64 KiB from 1 MiB through 64 MiB assigned ranges. Its G4.2 diagnostic then created a new backend and reader for each sample and fetched one manifest, one selected index, and one data block. As the complete range closure grew 63.96x, candidate bytes grew 1.19x and p99 grew 1.03x, reaching 506.459 microseconds at 64 MiB. The matched full-restore control transferred 860.59x more bytes and was 276.11x slower at p99. Both diagnostics use a dirty source tree and local filesystem cache with OTel disabled, so they establish an implementable read shape rather than an admitted production curve.

G4.3 composes that bounded object path with a retained tail and operating-system process replacement. Three OpenRaft authority processes selected generation 7, the publication root, and a logical txLog root. The first worker recovered the object base and three post-object mutations, then was killed before reading. A distinct empty-scratch replacement repeated recovery and returned exact base, update, delete, and tail-only insert reads. The lazy path reached 6.542 ms p99 and transferred 35,811 object bytes. Full hydration was 1.68x slower and moved 29.96x more bytes. Omitting the required tail produced nine anomalies. The tail adapter synchronizes three files on one machine, so this result establishes local composition, not a production replicated data log or machine-loss durability.

G4.4 replaces that adapter with a linearizable retained-transaction stream owned by three real OpenRaft data-authority processes. Accepted transaction commands are retained in state-machine snapshots and paged by commit version; workers never open a physical Raft journal. After the replacement caught up from object frontier $O = 9$ through $C_0 = 12$, the controller committed four more transactions through $C_1 = 16$. A second frozen catch-up returned exact point set, point clear, tail insertion, and range-clear outcomes. Across three seeds, the candidate reached 120.183 ms p99 from worker entry and transferred 6,177 object bytes. Full hydration was 1.64x slower and transferred 6.85x more bytes. Omitting the concurrent suffix produced twelve anomalies. The debug build, dirty source, barrier polling, single host, and disabled OTel keep this result Evaluating.

G4.5 then tests whether perfect objectification would bound that authority. The workload holds 256 live 128-byte values while lifetime commits grow from 256 to 4,096. A non-mutating projection removes every recovery command through $O = C$ before serializing the complete state. The projected snapshot still grows from 280,547 to 2,573,288 bytes, a 9.172x maximum across three seeds against a frozen 2.0x ceiling. The no-pop control grows 12.005x. A retained-stream-only poison reports a flat 1.0x curve but is rejected with nine incomplete-accounting anomalies. The current state shape is therefore discarded. Live values, OCC history, retry outcomes, and recovery commands require independent owners and reclamation frontiers before safe pop can be meaningful.

G4.6 implements that separation without mutating the object frontier. Latest values, OCC history, transaction retry records, and recovery commands become independently accounted owners. At each checkpoint, the candidate advances the minimum admitted read version R and the workload client's retry floor Q, then projects $O = C$. Complete projected bytes stay between 130,671 and 131,047, a 1.0029x maximum. Projecting only $O = C$ grows 9.1694x and nearly reproduces G4.5. A serving-only poison reports 1.0028x but is rejected with nine incomplete-accounting anomalies. Transactions below R and retries below Q fail before mutation, while all 4,096 recovery commands remain readable because O is still nonmutating. The architecture indicator is positive, but the candidate

takes 545.726 seconds against a 180-second execution budget. The split fixes retention economics, not commit-path speed.

G4.7 makes the projected object frontier physical. A publication quorum first retains one exact immutable row manifest as pending state. The controller walks and validates every manifest, sparse index, and row-data block before the data quorum removes recovery records through the declared coverage. Data voters then sign the exact frontier, active generation membership, and applied log position; publication activation requires a distinct quorum. Across three seeds, the candidate popped all 16 records through floor 18, recovered every value from objects, replaced both authority leaders, and restarted the killed data voter. The full protocol reached 160.331 ms p99, while physical pop took 20.519 ms at the median. Missing pending state and forged coverage failed before pop. A one-signer certificate failed activation after pop while the pending manifest remained protected and object recovery stayed exact. This closes the application recovery-stream invariant on one host. It does not purge the internal OpenRaft journal, prove independent-media durability, or improve the foreground commit rate.

G4.8 tests the least invasive commit-rate improvement before adding a new command format. A bounded proxy window submits up to 32 independent transactions concurrently; every transaction retains its own request identity, Raft entry, commit version, conflict result, and retry outcome. Across three release-build seeds, exact final values, retained-stream order, exact retry, leader replacement, and killed-voter restart remained correct. Median durable throughput rose from 38.772 transactions per second in the sequential control to 153.708, a 3.964x gain. The candidate nevertheless missed its frozen 200 transaction per second and 4x gates, and maximum p99 was 264.887 ms against a 250 ms ceiling. Stable-log observations explain the result: followers grouped entries into multi-entry appends, while the leader still performed one append for each transaction. An early-ack poison appeared to reach 3,400.389 transactions per second but lost the acknowledged transaction after quorum recovery. The suite rejected it. The next performance experiment must test an explicit commit-proxy batch entry rather than rename pipelining as group commit.

G4.9 implements that batch entry. Up to 16 independently fingerprinted client transactions share one Raft application entry and scalar commit version. A 16-bit batch order makes each transaction versionstamp unique. The state machine resolves items in encoded order, retains one result and recovery record per request, and pages the recovery stream by the versionstamp pair. Across three release-build seeds, the candidate reached 559.511 median durable transactions per second and 34.016 ms maximum p99. The same executable and stable journals with one entry per transaction reached 151.944 transactions per second and 262.174 ms maximum p99. The 3.682x paired gain cleared its 2.5x gate, and every candidate seed represented 16 logical transactions per leader append. Exact individual and whole-batch retry, in-batch conflict rejection, pair-valued pagination, final values, leader failover, and killed-voter restart passed. A duplicate identity failed before mutation. An early-ack batch appeared to reach 13,179.572 transactions per second but vanished after recovered-quorum election and was discarded. This admits the local mechanism for further stress, not a production commit curve.

G4.10 adds the client-facing boundary that G4.9 omitted. Independent request tasks enter a bounded FIFO queue. The proxy closes one batch on item count, exact encoded bytes, a 2 ms deadline, or sender shutdown, then validates result count and identity before completing each caller. Queue-full and oversized requests fail before replication. The first 16-item, 64-caller configuration reached 581.791 median transactions per second but was discarded because 131.488 ms maximum p99 missed the frozen 100 ms ceiling. Its 32-caller control reached 63.398 ms maximum p99, identifying a local admission knee.

A separately frozen 32-item candidate at 64 callers reached 1,157.369 median transactions per second and 76.101 ms maximum p99. The one-entry same-durability control reached 182.093 transactions per second, making the paired gain 6.356x. Every candidate seed used 32 logical transactions per leader append. Sparse traffic closed one-item batches on delay. The byte control closed at eight 8 KiB-value transactions and 119,731 application bytes under a 128 KiB cap. Bounded overload admitted and resolved 32 of 512 attempts and rejected the remaining 480 before replication. One oversized request was rejected before admission and mutation.

The byte control also exposed and removed a bootstrap artifact. Legacy JSON integer arrays expanded one 8 KiB value into an 89,097 byte entry. New `OKVT2`, `OKVQ2`, and `OKVB2` writes use unpadded base64 for opaque bytes while retaining v1 decoders and semantic retry identity. This is a backward-readable correction, not a final binary wire. The 32-item result remains EVALUATING because the tree was dirty, OTel was disabled, and all voters and journals shared one host.

G4.10b implements the RFC-0035 composition. It holds the 32-item path constant while a deterministic 25% conflict suffix overlaps authenticated frontier advancement through frozen O. Across three release-build seeds, the candidate reached 1,075.343 median resolved durable outcomes per second, 104.274 ms maximum p99, 31.030 minimum outcomes per leader append, and 95.673 ms maximum frontier time. The same-durability one-entry control reached 37.369 outcomes per second, making the paired gain 28.776x. The no-conflict control reached 1,093.306 outcomes per second, and the 75% conflict control remained bounded.

Every run reconstructed final C exactly from objects through frozen O plus retained transactions $(O, C]$. Exact committed and conflicted retries, both authority leader failovers, one killed-voter restart, and fresh-client recovery passed. A moving-frontier poison and an unprotected-pop poison left the retention floor unchanged and retained every prefix record. This admits the local composition mechanism, not a production cell. The receipt remains EVALUATING because the source was dirty, OTel was disabled, and all six processes, journals, and object files shared one host.

G4.11a closes the next local crash-safety seam. Each data voter writes a versioned, checksummed state snapshot to a same-directory candidate, synchronizes it, atomically replaces the authoritative file, and synchronizes the parent. Process purge fails unless that local snapshot covers the target. OpenRaft purge then canonical-rewrites the node journal to one vote, committed marker, purge marker, and retained suffix. Across three seeds, three journals fell from at most 6,391,575 bytes to 879 bytes. Stopping all three voters and reopening snapshot plus suffix preserved exact authority state, retained records, retry outcomes, and a new post-restart commit.

The poison requested purge without a covering snapshot; no purge marker, compaction count, or journal byte changed, and restart remained exact.

The same result falsifies the unfrontiered snapshot shape. Three snapshots totaled at most 5,066,472 bytes for 131,072 logical workload bytes, a 38.66082x maximum physical amplification. The journal is no longer the dominant growth source. Lifetime retry and recovery state are.

G4.11a.1 then executes four cycles that align resolver floor R, a 64-request retry window $Q(client)$, and authenticated object frontier O before each snapshot and purge. Across three seeds, candidate `be53d36c` preserves exact retries, full restart, and object-plus-suffix reconstruction with zero anomalies. Its 1.091759x snapshot growth passes the 1.25x gate, but its 19.692719x complete physical media misses the frozen 8x gate. The no-Q control reaches 54.803467x media and 2.195933x growth. An accounting poison reports 0.05365x by omitting snapshots, while the independent oracle recomputes 19.692719x. The frontier protocol is retained; the replicated snapshot representation is not admitted.

Before independent-media execution, the next architecture fork compares the row-object base with a manifested multi-layout LSM. The candidate uses row-oriented L0 delta runs, random-access columnar compacted runs, a kernel-owned primary access path, and one authenticated object closure. Its source of truth is the manifested closure through O plus *txLog* suffix $(O, C]$, not a file-format brand. Point, scan, ingest, compaction, media, HTAP, and branch curves decide whether one object state can replace separate transactional and analytical bases.

A 1,024-key local preflight keeps all three histories exact. Parquet scans at 1.873× the row control but uses 10 point requests and 16.35× its bytes; the hybrid uses four requests and 1.925× its storage. This rejects plain Parquet as the generic point format, not random-access columnar or typed-sidecar candidates.

The follow-up freezes a split typed-run subject: one indexed sidecar stores the complete MVCC value, while one columnar object stores only declared analytical fields. A release-local, three-seed, three-repeat comparison alternates subject order and passes every configured gate. Relative to the indexed-row control, the candidate uses 1.000× point requests, 1.000× point bytes, 1.033× median point p99, 9.124× projected-scan throughput, 1.030× stored/live amplification, 1.035× compaction write amplification, and 1.137× resident index bytes. This admits the mechanism to clean GCS evaluation, not to the opaque-KV default. Namespaced GCS execution and a frozen remote admission suite are now code-complete. No GCS receipt is claimed for this typed-run subject. The later GCP R0 receipt covers the public single-range row path and does not retroactively admit the C5 columnar layout on GCS.

The next subject removes the row sidecar. It stores MVCC keys, versions, visibility, and typed fields in checksummed projection stripes, stores opaque values once in independently checksummed payload pages, and keeps only a compact key-range index resident. A disposable RangeEngine cache is bounded to 16 MiB in the frozen profile. Release-local run `49d6cd06` again alternates subject order over three seeds and three repeats. Relative to the indexed-row control, cold points use 1.982× requests, 0.353× response bytes, and 0.839× median p99. Projected scans reach 4.718× throughput while reading zero opaque payload bytes. Stored/live, compaction-write, and resident-index ratios are 1.010×, 1.010×, and 1.170×. Repeated warm reads issue zero object requests, and a fresh process reconstructs the exact MVCC history from the manifest, index, projection, and payload objects. The source is dirty and telemetry is disabled, so this admits a mechanism to DataFusion and GCS evaluation, not a verified architecture.

The next experiment replaces the in-memory or Parquet base source with a custom DataFusion `TableProvider` and incremental `ExecutionPlan` over the same projection stripes. Point and scan reads use different physical request sizes. A point retains one approximately 7.8 KiB stripe. A scan coalesces adjacent stripes into an independently verified range no larger than 256 KiB, then emits the original batches with at most 128 rows each. Dirty release-local run `a7d4f3bf` reaches 2.544 million source rows per second with 54 projection range GETs across nine samples. Same-suite one-stripe control `d788c75f` reaches 1.247 million rows per second with 1,761 GETs. The candidate therefore reaches 2.040× throughput with 0.0307× requests and identical projection bytes. Its maximum raw fetch buffer is 257,506 bytes and maximum Arrow batch is 1,646 bytes. It reads no opaque payload page. Payload-prefetch poison `b4fe7c11` adds 1,761 requests, triples transferred bytes, and reduces throughput to 0.820 million rows per second. This result establishes the direct source and dual-granularity fetch mechanism. It does not establish complete-query memory, base-plus-live-tail execution, or GCS latency.

CloudJump III provides a production comparison for the tiering mechanism, but not for the C5 physical layout. It integrates InnoDB page placement at eviction and flush across DRAM, a volatile local-SSD buffer-pool extension, a durable network-block OSS Buffer, and versioned 2 MiB object writes [5]. WAL protects dirty pages until they reach the durable tier. The OSS Buffer then combines 16 KiB page updates before asynchronous publication. Its experiments sweep 5–50% fast-tier coverage and Zipf skew from 0.8 to 2.0; near-local throughput appears primarily after the reusable hot set fits a moderate fast tier. This supports independent point, scan, and publication geometry plus engine-aware cache admission. It does not establish a columnar source of truth, exact HTAP execution, or the sufficiency of retained *txLog* mutations without a durable page-image buffer.

We implemented the first cache-policy ablation implied by this comparison. At 20% cache coverage and Zipf 1.4, dirty release-local runs over three seeds and three repeats give ghost two-chance a 74.46% post-scan hit ratio versus 71.34% for full admission, while reducing post-scan object requests by 16.2%. A one-seed canary through the real GCS adapter gives 42.19% versus 32.03%, cuts post-scan requests from 161 to 128, and reduces wall time from 75.06 to 67.14 seconds. Exactness and cache-capacity gates pass. Dirty source, disabled OTel, one cloud seed, and the small canary keep both receipts inconclusive.

7 SYSTEMS TO INFRASTRUCTURE

Figure 6 answers whether current work runs on real infrastructure or mocks. The answer is mixed and deliberately scoped.

The deterministic contracts are executable models. They provide exact replay, state-space control, and negative-subject detection,

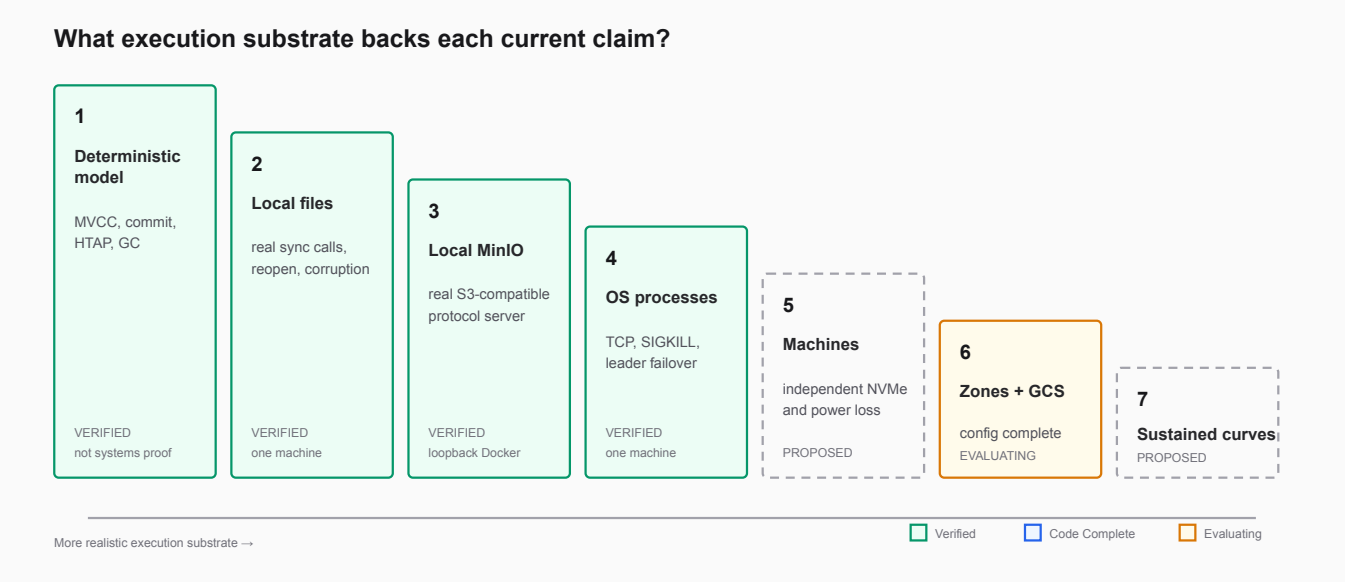

Figure 6: The systems-to-infrastructure proof ladder. Current evidence includes one sequential source-media-loss reconstruction, but remains below simultaneous independent-voter and multi-zone verification.

but no operating-system or device claim. The stable-file lanes use real files, checksums, synchronization, corruption, and reopen on one laptop. The MinIO lane uses a real pinned S3-compatible server over loopback. The process lanes use real executables, TCP sockets, SIGKILL, replacement processes, and OpenRaft election. Turmoil provides deterministic network and crash simulation and is not a real network.

G4.4 removes the local G4.3 tail adapter and proves a bounded two-round catch-up while commits continue. G4.5 shows that popping only that retained stream cannot bound the monolithic authority snapshot. G4.6 splits the four retention domains and keeps complete projected state flat, but misses its execution budget. G4.7 authenticates and mutates O through a pending immutable closure and data-voter quorum certificate. G4.8 then shows that concurrent one-entry submission pipelines the current path but leaves leader synchronization proportional to transaction count. G4.9 removes that local bottleneck with an explicit batch entry. G4.10 then starts from independent requests, discards the 16-item configuration on p99, and retains a separately frozen 32-item local candidate after its delay, byte, overload, absolute, and paired gates pass. G4.10b joins that path to authenticated objectification while commits continue, preserving exact object-plus-suffix recovery and clearing its absolute and paired local curves. G4.11a adds crash-safe process snapshots and physical journal reclamation, then exposes 38.66082x amplification in the unfrontiered snapshot. G4.11a.1 proves that real R, $Q(client)$, and O transitions bound the lifetime curve, but rejects the current representation at 19.692719x complete media against an 8x gate. The next local gate compares compact row, columnar, and multi-layout manifested runs before expanding the recovery claim to cloud hosts. It must also bound distinct-client cardinality, retained bytes, and maintenance interference as $C - O$ and write rate increase.

The next infrastructure slice is one GCP project, one protected single-region GCS bucket, three compute instances with independent stable-log media, a collector, and a workload runner. NVMe is the first durability control, not a requirement that every serving profile use an SSD image. The first pass should remain one region so it can isolate machine, log media, serving profile, object, and zone effects without claiming a multi-region product. Required experiments include:

- direct NVMe synchronization and same-durability Raft controls;
- GCS immutable PUT, conditional authority, range GET, unknown response, and request-cost profiles;
- process, machine, disk, and zone loss during commit and publication;
- empty-worker replacement from objects plus retained *txLog* for both SSD-resident and RAM-resident serving profiles;
- object-store throttling and outage until warning, rate-limit, and refusal thresholds activate;
- sustained resident, cold-read, commit, objectification, and cost curves with exact OTel receipts.

The project, regional versioned GCS bucket, VPC, service account, and required APIs were observed through GCP on August 26, 2026. The isolated R0 Terraform root defaults to zero compute resources. On August 27 it produced a machine receipt for one private `n2-standard-8`, a 200 GiB persistent `pd-ssd`, a 375 GiB local NVMe SSD, and a separate private OTel collector. Candidate and control receipts share the machine, source, lockfile, seeds, and batch identity. OTel retained logs, metrics, and traces for both. All nine leased resources were destroyed after evidence capture.

8 EVALUATION PLAN AND TARGET CURVES

The evaluation program has one primary metric per lane, frozen correctness oracles, immutable seeds, negative controls, machine and backend profiles, and OTel receipts. It does not optimize one blended score. A candidate is eligible for performance comparison only after correctness gates pass.

The comparison authority is lane-specific. A direct RocksDB run is a serving mechanism control, not a durability-equivalent system control; a matched three-voter TiKV deployment is the later quorum-commit system control. The code-complete paired comparator rejects mismatched metric, statistic, direction, unit, batch identity, machine, toolchain, lockfile, source revision, seed, suite/profile identity, failed hard gates, and fewer than five performance samples. For a higher-is-better metric it reports $(x_c - x_i)/|x_i|$; for a lower-is-better metric it reports $(x_i - x_c)/|x_i|$. A result is decisive only when the effect clears both the declared practical threshold and the larger candidate/control MAD-to-median ratio. The receipt is otherwise inconclusive or invalid.

The code-complete golden scenario freezes one deterministic commerce-ledger history across 12 surfaces and 15 dependency-ordered checkpoints. Its artifact graph begins with semantic history, passes through the durable tail and object base, branches into SSD-resident and RAM-resident serving, rejoins at fenced profile handoff, and continues through empty-worker recovery, branching, multi-range transactions, application logs, Redis, search, PostgreSQL, exact HTAP, and economics. Every checkpoint is attached to a typed gate, but none yet has a verified end-to-end receipt. Existing component receipts remain evidence for their named components rather than being retroactively promoted into golden-path completion.

A smaller paired developer scenario is independently verified at its declared scope. Tetris retained 2,524,571 bytes of materialized *txLog* payload for 2,000 actions. Two-byte deltas plus eight encoded checkpoints retained 5,904 logical bytes and reconstructed the same fingerprint, a 427.6x ratio. The active Chess materialized branch retained 3,155 bytes; three four-byte deltas plus one encoded checkpoint retained 126 logical bytes and restored the same divergent fingerprint across both switches and replay, a 25.0x ratio. Logical bytes exclude durable frames, indexes, replication, and object metadata. In the release harness, RAM point-read p99 was 125 ns for Tetris and 84 ns for Chess; these in-process measurements exclude RPC, concurrency, and cache misses. Both workloads published a branch with two new objects, copied no immutable prefix, reclaimed exactly two child-only objects, and reopened the main root exactly. The next gate composes replicated authority, real GCS, and comparable RAM and SSD serving profiles under the same frozen histories.

The SSD hot-read and commit curves determine whether objectKV can approach a TiKV/RocksDB service class. The RAM curve determines whether a second serving profile earns its capacity and recovery tradeoff rather than merely moving the same work into a more expensive medium. The cold-read and reopen curves determine whether object separation is real rather than a renamed full replica. Objectification and brownout curves determine whether asynchronous permanence stays bounded. The HTAP curve determines whether exact base-plus-tail execution is useful in practice. The economics curve selects which implementation owns each layer. If native transaction authority fails its frozen gates, the project should select FoundationDB or another admitted incumbent behind the same objectKV log, object, history, and fabric boundaries rather than stop the program.

9 DESIGN TRADEOFFS AND ALTERNATIVES

9.1 Why not FoundationDB plus object storage?

FoundationDB already offers mature ordered transactions and independent storage processes. Using it minimizes transaction-system risk. A derived object materializer could provide snapshots and analytics. The cost is that permanent storage, recovery topology, version retention, and object identity remain adaptations around a kernel designed for replicated storage servers. objectKV owns the reconstruction equation and object publication boundary directly. That may improve portability and branching, but it also assumes responsibility for consensus, conflict resolution, recovery, data distribution, and years of failure hardening.

9.2 Why not TiKV on RocksDB?

TiKV is the strongest default for a distributed transactional KV over RocksDB. It provides a credible MultiRaft path and powers TiDB. A TiKV-based design could export cold data or analytical artifacts to objects. This minimizes novel hot path work. It does not make object reconstruction the authoritative permanent base, and full replicas still define placement and restore behavior. objectKV should borrow this hot-path shape, including RocksDB serving state and range-local consensus, while proving that object-backed rebuild and retention change something material.

9.3 When can columnar objects be the permanent source of truth?

Apache Paimon shows that a primary-key table can organize columnar objects as an LSM and merge overlapping sorted runs at read time [3]. Its PFile proposal also records a boundary: dynamically converting columnar files for KV lookup did not scale to the desired lookup-table sizes, which motivated a dedicated KV-oriented file [2]. RisingWave similarly uses its row-based Hummock LSM for point reads and frequent updates while retaining columnar storage for analytical workloads [13].

Newer random-access columnar formats make this boundary worth measuring rather than assuming. Lance combines immutable manifests, fragments, deletions, row addressing, and independent indexes [12]. Vortex exposes extensible columnar layouts with explicit zone and chunk organization [17]. SAP HANA provides the stronger precedent: a write-optimized L1 delta, a columnar L2 delta, and a compressed columnar main share one table interface, stable row identity, and secondary indexes for point access [15]. HANA's Native Storage Extension later makes column primitives independently resident or page-loaded behind a bounded cache [14].

The corresponding objectKV candidate keeps logical truth as $ManifestedObjectState(O) + txLog(O, C]$. The bounded mutable delta serves recent writes and conflicts. Immutable column primitives

own permanent state. A dense key-to-row index selects one metadata stripe; the point path then gathers one opaque-value page, while scans stream only projected stripes. DataFusion consumes those stripes as bounded Arrow batches rather than using Parquet as a point store [8, 9].

CloudJump III establishes a complementary lesson for tier management rather than columnar layout. Its engine-visible ghost admission, per-table quotas, temporary-data exclusion, write combining, and large-write bypass show that a generic cache cannot infer every useful placement decision from block access alone [5]. ObjectKV therefore keeps SQL semantics out of the transaction kernel but evaluates bounded range-level placement classes above it. Those classes may affect cache and publication policy, never transaction visibility or durability.

Local run `49d6cd06` is the first mechanism evidence that this can beat the row control on bytes, p99, scans, and total media simultaneously. It does not prove remote latency, arbitrary PostgreSQL schema evolution, concurrent delta merge, or DataFusion pushdown. The split row-sidecar layout remains the fallback if the second remote object request dominates GCS point latency.

9.4 Why not an object-only LSM?

An object-only WAL can simplify infrastructure for single-writer or latency-tolerant workloads. It conflicts with the target multi-writer commit latency and failure model. Batching trades requests for latency, while scaling writers introduces fencing and replicated coordination. The chosen durable profile pays for a stable replicated *txLog* explicitly and keeps object publication asynchronous. NVMe is its first implementation control. A weaker RAM-only profile must expose a weaker outcome rather than redefining commit.

9.5 Row versus page values

The kernel is value native. PostgreSQL can begin page native to preserve the upstream engine and reduce the initial semantic rewrite. A later row-native adapter could exploit distributed transactions and analytical change indexes more directly, but it would own much more PostgreSQL behavior. The project does not need to decide one permanent representation before measuring the page bridge and its double-logging cost.

9.6 Transaction-plane controls

The matched native resident boundary passed its frozen throughput and p99 envelopes through 32 clients. It remains the primary implementation lane. The unmatched failure is retained as evidence that controls must pay for the same recovery topology and owned-value semantics. FoundationDB separately owns the strict-serializability and lifecycle control while ObjectKV develops retained history, immutable object closures, reconstruction, and generation fencing. The current bounded control data flow is:

```
FoundationDB source S
       | exact closure at C
       v
named GCS manifest + closure
       | delete S VM and both disks
       | controller observes absence
       v
fresh FoundationDB destination D
       | idempotent chunks
       v
exact digest -> activation -> fresh commit
```

Figure 7: The verified GP2.5.3 media-loss path. The object closure crosses the source-destination boundary; no source provider state does.

The frozen positive run captured 950 rows in a 205,262-byte closure, verified named GCS reads, restored five chunks twice without changing state, matched the source digest, and rejected a transaction that began before generation activation. Three poisons for missing retained change, missing request outcome, and missing generation dependency were detected and received discard verdicts. The positive probe took 512.560 ms internally and 840.212 ms end to end. These are single-sample diagnostics. Every final semantic and lifecycle run ID occurs in captured OTel logs, metrics, and traces.

GP2.5.3 then froze one source and one destination provider identity. The source wrote a 205,248-byte closure plus a 607-byte manifest for 950 final records. Terraform deleted the source VM, boot disk, and 100 GiB provider SSD; an external controller observed all three absent 227 seconds before restore began. The fresh destination read only the named GCS generations, restored and replayed five chunks, reproduced the exact row count and state digest, activated after its ready marker, and accepted one fresh transaction. The formal positive received `keep` with 16 passing gates and zero anomalies. The same-cluster hidden-source-media control received `discard`. Both run IDs occur in all three captured OTel signals. The 305.667 ms destination lifecycle is a single-sample diagnostic and does not establish HA, incarnation fencing, RPO, RTO, or performance.

GP2.5.4 now has a clean local-process receipt for the compound authority. The positive started twelve authority or publication processes, six data processes, killed seven processes, crossed three authority failovers, rejected four fenced commit attempts, and retained valid destination publication. The stale-source control bypassed commit, route, and publication fencing and was discarded with exactly three anomalies. The remaining GP2.5.4 claim is the provider composition: a source FoundationDB fence must persist across a VM restart after a distinct destination activates. The code-complete GCP harness retains both provider disks, orders source fence before destination activation, restarts the source without replacing its media, and emits a strict receipt. Activation binds the exact source-fence receipt digest, while resurrection binds the activation and restart receipt digests. Cross-provider wall clocks are diagnostic rather than admission inputs.

10 LIMITATIONS AND OPEN QUESTIONS

The current implementation does not yet support a full client transaction from read version through multi-range commit and routed read. It has no admitted RAM-resident serving profile, stable complete transactional segment format, compactor, range splitter, data

distributor, multi-range commit protocol, PostgreSQL adapter, application-log service, or snapshot manifest and lease service. Its cloud receipts remain bounded R0 runs rather than a production deployment.

Five questions can still invalidate the architecture:

(1) Can a bounded local serving image produce stable hot p99 without retaining the complete durable database on every serving replica?
(2) Can cold lookup metadata and object requests remain bounded as total history and database size grow?
(3) Can range-local consensus scale writes while cell-level transactions remain strictly serializable and recover without partial outcomes?
(4) Can objectification and garbage collection stay bounded under object brownouts, retries, branches, leases, and slow analytical materialization?
(5) Does at least one target workload gain enough branch, recovery, HTAP, footprint, or cost advantage to justify a new transaction kernel?

The system also gives up cross-cell transactions, unbounded transactions, arbitrary server-side computation inside the kernel, and object-only low-latency commits. These are explicit product constraints rather than temporary implementation omissions.

11 CONSTRUCTION ROADMAP

The dependency order is evidence driven:

(1) Decompose the sequential commit path and compare bounded transaction batching with one-entry-per-transaction control under the same stable-media durability contract. Keep the native authority only if correctness holds and the throughput curve clears its frozen gate.
(2) The topology-matched native SSD-resident experiment passed exact-read, object-request, local-byte, throughput, and p99 gates in both process orders. The 8-client and 32-client curves also passed. Next freeze cache pressure and native replicated commit while FoundationDB remains the semantic oracle.
(3) The GCS closure passed empty logical-generation reconstruction and fresh-provider reconstruction after source VM and disk deletion. The local external incarnation authority is verified and the real-provider harness is code complete. Next execute the source-fence, destination-activation, and source-resurrection receipt, then compare mandatory retained-write overhead against direct FoundationDB.
(4) Verify generation-fenced SSD-to-RAM and RAM-to-SSD handoff without stale reads, stale publication, or lost committed state.
(5) Define and verify the indexed row-object format, bounded cold lookup, asynchronous packing, and compaction economics.
(6) Run the existing commit and publication contracts on independent GCP machines, matched persistent SSD media, and GCS, including machine and zone failure.
(7) Join the paths into one Cell v0 vertical slice with one range group, then add range-local groups and cell-level transaction atomicity.
(8) Add the PostgreSQL page bridge and `okv-log` based application stream plus PostgreSQL WAL adapter, with complete byte, retention, and synchronization accounting.
(9) Add version-bound analytical manifests and leases, then measure the exact DataFusion tail curve.
(10) Evaluate total economics against TiKV and PostgreSQL plus object tiering. Stop owning the kernel if no target workload wins materially.

This sequence intentionally delays the impressive distributed surface until the resident and object economics clear. It also keeps PostgreSQL and ZebraDB as consumers rather than using them to conceal missing kernel properties.

12 RELATED WORK

FoundationDB's transaction system, resolvers, logs, and storage servers are the closest semantic reference for the cell architecture [18]. Raft provides the initial replicated-log protocol and understandability baseline for the implementation [11]. TiDB and TiKV demonstrate a practical MultiRaft transactional row store and an HTAP row-to-column path [7]. RocksDB supplies the first resident serving-engine control [6].

Aurora separates a database engine from a distributed storage service and offloads redo processing [16]. Socrates decomposes SQL Server into compute, log, page, and storage tiers [1]. Neon separates PostgreSQL compute, a quorum log service, and object-backed page reconstruction, which closely motivates the PostgreSQL path [10]. objectKV differs by seeking a general ordered transactional kernel below PostgreSQL and by making row and analytical objects projections of one value-native history.

DataFusion provides the extensible table-provider and execution-plan interfaces used by the exact analytical overlay [8]. Apache Parquet and Arrow provide the current column and memory formats [4, 9]. The analytical architecture is closer to a version-aligned base plus change tail than to a separate warehouse replica.

13 CONCLUSION

objectKV is plausible only as a hybrid system: a stable replicated *txLog* for durable commits, bounded SSD or RAM profiles for resident reads, and immutable objects for the reconstructable permanent base. A deliberately weaker RAM-only durability profile may be useful, but it cannot claim committed data survives total volatile-state loss. The current program has crossed the line from pure models into real local files, a real S3-compatible server, real OpenRaft processes, TCP, process kill, failover, replacement recovery, regional GCS, and a private local-NVMe runner. It has not crossed the line into independent data machines or zones, a full transaction cell, or product-level performance.

The application-delta result proves a compact logical representation, not a durable encoding. Its next rung must append each delta transactionally, recover through `okv-wal`, bind checkpoints to exact positions and reducer identities, and reject corrupt or unsupported histories before serving state.

The native resident boundary has supplied two decisions. First, comparisons must match recovery topology and value ownership. Second, once matched, the native version-bound read boundary stays inside the frozen direct RocksDB envelope through 32 clients.

The native plane therefore continues to cache pressure and replicated-commit gates. TiKV failed the strict-serializable write-skew control; FoundationDB passed and remains the semantic oracle and fallback profile. Its logical-generation and media-loss probes verify exact object reconstruction in their bounded scopes, while the local compound-fence process detects stale commit, route, and publication attempts. No receipt yet proves a complete native transaction cell, independent-zone quorum durability, cache-pressure behavior, PostgreSQL execution, or end-to-end HTAP economics. The golden scenario is code complete as a plan. The next decisive results are the bounded cache curve, same-durability native replicated commit, and exact object-plus-tail recovery under those admitted rates.

REFERENCES

[1] Panagiotis Antonopoulos, Alex Budovski, Cristian Diaconu, Alejandro Hernandez Saenz, Jack Hu, Hanuma Kodavalla, Donald Kossmann, Sandeep Lingam, Dinesh Nica, Parikshit Purohit, Peter Qu, Chaitanya Ramesh, Ashay Ravi, Cristian Simion, et al. 2019. Socrates: The New SQL Server in the Cloud. In *Proceedings of the 2019 International Conference on Management of Data*. 1743–1756. https://doi.org/10.1145/3299869.3314047

[2] Apache Software Foundation. 2024. PIP-25: Introduce a Key-Value File Format for Paimon Primary Key Table. https://cwiki.apache.org/confluence/pages/viewpage.action?pageId=311628201

[3] Apache Software Foundation. 2026. Apache Paimon Primary Key Table Overview. https://paimon.apache.org/docs/1.1/primary-key-table/overview/

[4] Apache Software Foundation. 2026. Apache Parquet Format. https://parquet.apache.org/docs/file-format/

[5] Zongzhi Chen, Mo Sha, Feifei Li, Sheng Wang, Baolin Huang, Guoqing Ma, Huaxiong Song, Ke Yu, Xizhe Zhang, and Yuan Wang. 2026. CloudJump III: Optimizing Cloud Databases for Tiered Storage. In *Companion of the International Conference on Management of Data*. 266–280. https://doi.org/10.1145/3788853.3803084

[6] Siying Dong, Mark Callaghan, Leonidas Galanis, Dhruba Borthakur, Tony Savor, and Michael Strum. 2017. Optimizing Space Amplification in RocksDB. In *CIDR*. https://www.cidrdb.org/cidr2017/papers/p82-dong-cidr17.pdf

[7] Dongxu Huang, Qi Liu, Qiu Cui, Zhuhe Fang, Xiaoyu Ma, Fei Xu, Li Shen, Liu Tang, Yuxing Zhou, Menglong Huang, Wan Wei, et al. 2020. TiDB: A Raft-based HTAP Database. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3072–3084. https://doi.org/10.14778/3415478.3415535

[8] Aviral Khurana et al. 2024. Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine. In *Companion of the 2024 International Conference on Management of Data*. https://dl.acm.org/doi/10.1145/3626246.3653368

[9] Wes McKinney Li et al. 2016. Apache Arrow: A Cross-Language Development Platform for In-Memory Data. In *Proceedings of the VLDB Endowment*. https://arrow.apache.org/

[10] Neon Contributors. 2022. Neon Architecture. Neon Documentation. https://neon.tech/docs/introduction/architecture-overview

[11] Diego Ongaro and John Ousterhout. 2014. *In Search of an Understandable Consensus Algorithm*. Technical Report. USENIX Association. https://raft.github.io/raft.pdf

[12] Weston Pace, Chang She, Lei Xu, Will Jones, Albert Lockett, Jun Wang, and Raunak Shah. 2025. Lance: Efficient Random Access in Columnar Storage through Adaptive Structural Encodings. *arXiv preprint arXiv:2504.15247* (2025). https://arxiv.org/abs/2504.15247

[13] RisingWave Labs. 2026. RisingWave Storage Overview. https://docs.risingwave.com/store/overview

[14] Reza Sherkat, Christian Florendo, Mihnea Andrei, Young-Seok Choi, Guk-Han Lee, Carsten Lemke, Nitin Pathak, Somdip Saha, Natalia Sokolova, Anatoly Stetsenko, et al. 2019. Page As You Go: Piecewise Columnar Access in SAP HANA. *Proceedings of the VLDB Endowment* 12, 12 (2019), 2047–2058. https://www.vldb.org/pvldb/vol12/p2047-sherkat.pdf

[15] Vishal Sikka, Franz F"arber, Wolfgang Lehner, Sang Kyun Cha, Thomas Peh, and Christof Bornh"ovd. 2012. Efficient Transaction Processing in SAP HANA Database: The End of a Column Store Myth. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*. 731–742. https://doi.org/10.1145/2213836.2213946

[16] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, Murali Brahmadesam, Kamal Gupta, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvili, and Xiaofeng Bao. 2017. Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 1041–1052. https://doi.org/10.1145/3035918.3056101

[17] Vortex Contributors. 2026. Vortex File Format. https://docs.vortex.dev/concepts/file-format

[18] Jingyu Zhou, Meng Xu, Alexander Shraer, Bala Namasivayam, Alex Miller, Evan Tschannen, Steve Atherton, Andrew J. Beamon, Rusty Sears, John Leach, Dave Rosenthal, Xin Dong, Will Wilson, Ben Collins, David Scherer, Alec Grieser, Young Liu, Alvin Moore, Bhaskar Muppana, Sampath Raghunath, Anirudh Rao, et al. 2021. FoundationDB: A Distributed Unbundled Transactional Key Value Store. In *Proceedings of the 2021 International Conference on Management of Data*. 2653–2666. https://doi.org/10.1145/3448016.3457559

Table 1: Selected implementation evidence as of 2026-08-27. A zero-anomaly result is scoped to the listed backend and topology.

Lane	Status	Execution substrate	Receipt	Primary result
MVCC and commit contracts	Verified	Deterministic model, frozen seeds and negative subjects	Multiple versioned suites	Zero anomalies in admitted seeds
Persisted WAL reopen	Verified	Three ordinary files on one machine, real synchronization and fresh opens	Local-fs suite	Matching contiguous two-file quorum only
Object authority profile	Verified	In-memory authority and pinned MinIO on Docker loopback	MinIO run `90b83f78`	12 of 12 checks
OpenRaft failover	Verified	Three real local processes, TCP, process kill, leader failover	Run `ce63...` and process lanes	Zero anomalies across three fixed seeds
Lost Publish response	Verified	Local object files plus OpenRaft processes, publisher and leader kill	Run `a544deff`	42 checks, zero anomalies, six kills
DataFusion overlay	Verified	Local Parquet and Arrow execution	Run `b239a722`	Exact ratio 1, peak 5,518 bytes, zero spill
Columnar RangeEngine overlay	Evaluating	Release local filesystem, bounded 16 MiB cache, three seeds and three alternating repeats, dirty source	Run `49d6cd06`	Point requests 1.982x and bytes 0.353x row control; p99 0.839x; projected scans 4.718x; stored/live 1.010x; zero warm requests
Direct DataFusion range source	Evaluating	Release local filesystem, bounded 256 KiB scan fetch, three seeds and three repeats, dirty source	Runs `a7d4f3bf`, `d788c75f`, `b4fe7c11`	2.544M rows/s; 2.040x one-stripe control; 54 versus 1,761 projection GETs; zero payload reads
Application history	Verified	GP-G0–G6 bounded local profiles	Paired ladder receipt	Exact replay, failover, RAM rebuild, object reopen, fork and GC; 427.6x Tetris and 25.0x Chess logical-size ratios
Serving-process recovery	Evaluating	Three OpenRaft authority processes, two worker processes, local quorum files	G4.3 receipts	6.542 ms p99 and 35,811 object bytes; full hydration 1.68x slower and 29.96x larger
Authenticated object frontier	Evaluating	Three publication plus three data OpenRaft processes, local files, dirty debug build	G4.7 runs `f314d6e1`, `e3467af2`, `7bbf3d7e`, and `5bfbd653`	16 records physically popped, exact object recovery, 160.331 ms protocol p99; three unsafe controls rejected
Independent-request commit proxy	Evaluating	Three OpenRaft data processes, release build, local stable journals, dirty source	G4.10a.1 runs `be37cc6b`, `cbe29754`, and five policy controls	1,157.369 tx/s, 76.101 ms maximum p99, 6.356x one-entry control, 32 transactions per leader append
Durable snapshot and frontiered compaction	Evaluating	Six local processes, release build, synchronized files, dirty source	G4.11a and G4.11a.1 runs	Journals fell to 879 bytes; exact restart; frontiered growth 1.091759x; 19.692719x media failed 8x
GCP single-range mechanism	Verified	Private n2-standard-8 runner, regional GCS, 200 GiB persistent SSD, 375 GiB local NVMe, OTel	Runs `662481a8` and `c571ff45`	Exact recovery, zero hot object operations, zero anomalies, full lease teardown
SSD public read admission	Evaluating	Same frozen GCP runner and 4 MiB warm working set, 15 samples per subject and order	Batches `gp31nvme2-ab-r1` and `gp31nvme2-ba-r1`	Throughput 19.32% and 19.98% worse; p99 35.25% and 29.99% worse; executable p99 constraint failed twice
Provider media loss	Verified	Sequential private source and restore VMs, distinct 100 GiB provider SSDs, regional GCS, OTel	Runs `3093a364` and `865c06d8`	Exact 950-record reconstruction after source VM and disk deletion; 16 positive gates passed; hidden-media control discarded
Provider incarnation authority	Evaluating	Three-process generation and publication authorities on one host; dual-provider GCP resurrection harness pending	Runs `15260d66` and `b05dd054`	Local positive kept with zero anomalies; stale commit, route, and publication poison discarded with three anomalies
Independent hosts and zones	Evaluating	Not yet executed	No admitted receipt	Single-runner evidence does not establish distributed durability

Table 2: Initial falsifiable target curves. Targets are proposed until a frozen control and machine profile are admitted.

Curve	Control	Initial target	Stop or redesign signal
Hot SSD point read	Direct RocksDB and TiKV-like resident path	At 99.9% resident coverage, p99 and throughput within 20% of control; zero object requests after warmup	Stable p99 requires a complete durable local database copy
Fast-tier policy	All-fast-tier same-durability control	Sweep 5–50% coverage, Zipf 0.8–2.0, and full versus ghost admission; report p99, hit rate, cache IOPS, and cost per operation	Near-local results require an unbounded cache, fixed skew, or hidden durability state
Hot RAM point read	Same history and durability on the admitted SSD profile	At least 20% end-to-end improvement in the selected latency or throughput metric under the same correctness gates	RAM capacity, recovery, or handoff cost erases the hot-path gain
Cold point read	Indexed local block and direct object range GET	Bounded GETs, bytes, decode, and metadata independent of total database size	One miss scans an object or walks history that grows with database size
Commit	Same-durability Raft or MultiRaft KV	One-range p99 within 25% of control; object operations absent from normal trace	Global coordination dominates or object storage enters commit latency
Scale-out	Single range-group control	Independent range groups increase committed transactions per second until a named resource saturates	One cell-global service caps throughput before range groups scale
Objectification	Direct batch uploader	Stable lag under admitted ingest and bounded retained log during brownout	Debt grows without bound or acknowledged state becomes unrecoverable
Local durability state	Unfrontiered snapshot plus append-only journal	At most 8x one logical copy after three-replica snapshot and purge; cycle-four snapshot at most 1.25x cycle one	Snapshot or journal grows with objectified lifetime history
Application delta	Materialized KV history	Exact differential fingerprint; bounded checkpoint tail; Tetris at most 4 and Chess at most 6 logical bytes/action over a complete interval	Reducer evolution or checkpoint identity makes retained history unrecoverable
Empty-worker reopen	Full local replica restore	First read depends on bounded metadata, assigned range, and requested blocks, not total cell bytes	Full-cell download is required before serving
HTAP overlay	Base-only DataFusion query	Tail at or below 1% adds at most 20%; materialization intervenes before 10%	Exact tail work dominates inside the admitted policy
PostgreSQL bridge	Same upstream revision on local storage	Within 2x for first prototype, then within 25% at resident steady state	Double WAL, page amplification, or recovery remains structurally noncompetitive
Economics	TiKV or PostgreSQL plus object tier	Material cost, branch, recovery, footprint, or HTAP advantage in at least one target workload	Higher cost without a material operational or product advantage